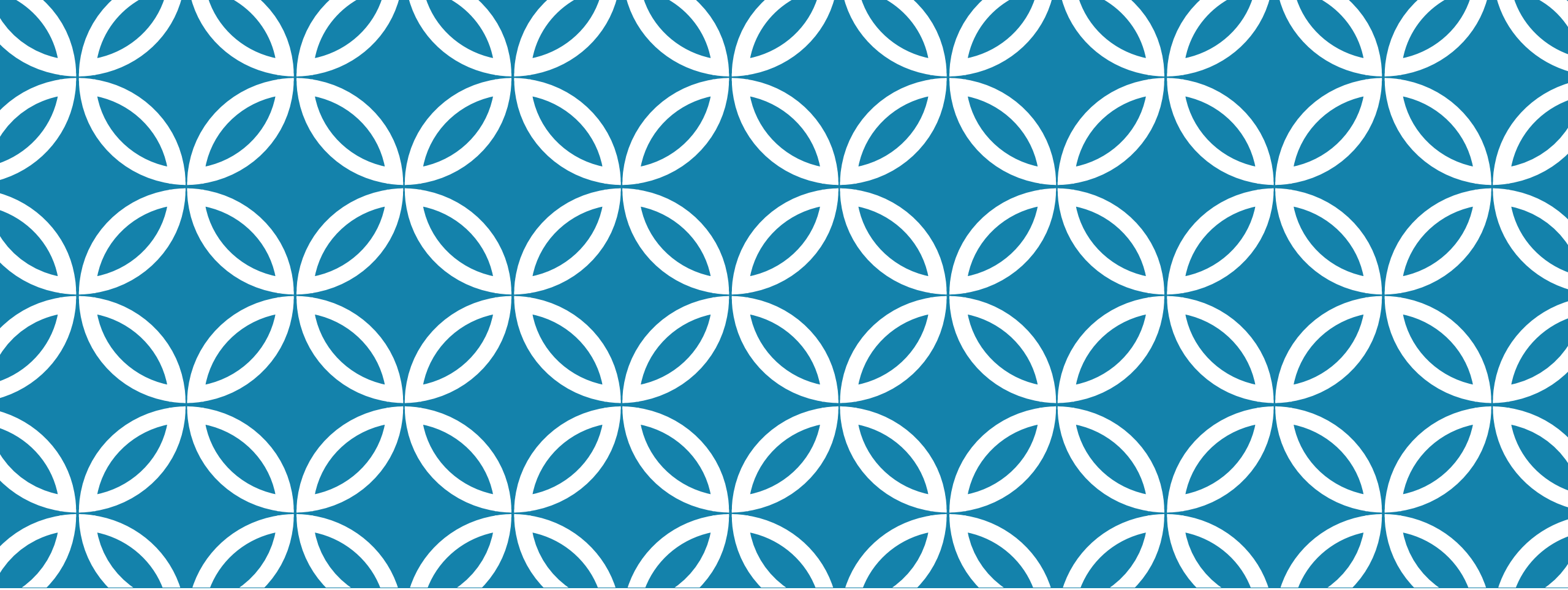


GREEDY ALGORITHMS

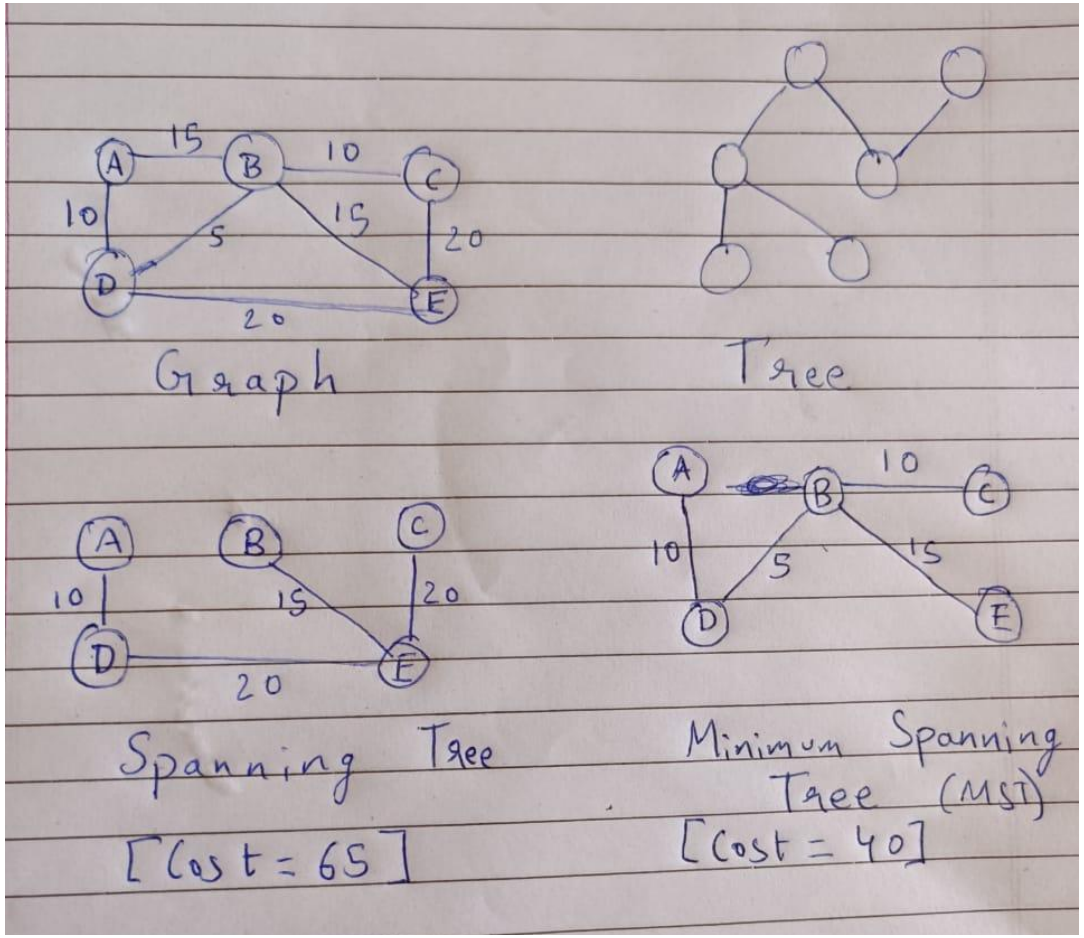
- By Ohm Patel

MINIMUM SPANNING TREE

- A tree is a special case of the graph. It is a graph that has no cycles and V-1 edges.
- A spanning tree is one that touches every node of the graph.
- A minimum-spanning tree (MST) is a tree that touches every node of the graph while having the overall lowest cost.
- An MST always has V-1 edges (where V is the number of vertices in the graph).
- We will look at 2 algorithms to find the minimum spanning tree for a graph:

    1. Prim's Algorithm
    2. Kruskal's Algorithm

MINIMUM SPANNING TREE



PRIM'S ALGORITHM

- Prim's algorithm always starts with a single node in a graph and moves through several adjacent nodes, in order to explore all of the connected edges along the way.
- The goal is to create a minimum spanning tree(MST) i.e. a tree that connects all the vertices with the minimum possible total cost.
- It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.
- The time complexity of this algorithm is O(Elog(V)) where V is the number of vertices and E is the number of edges in the graph.

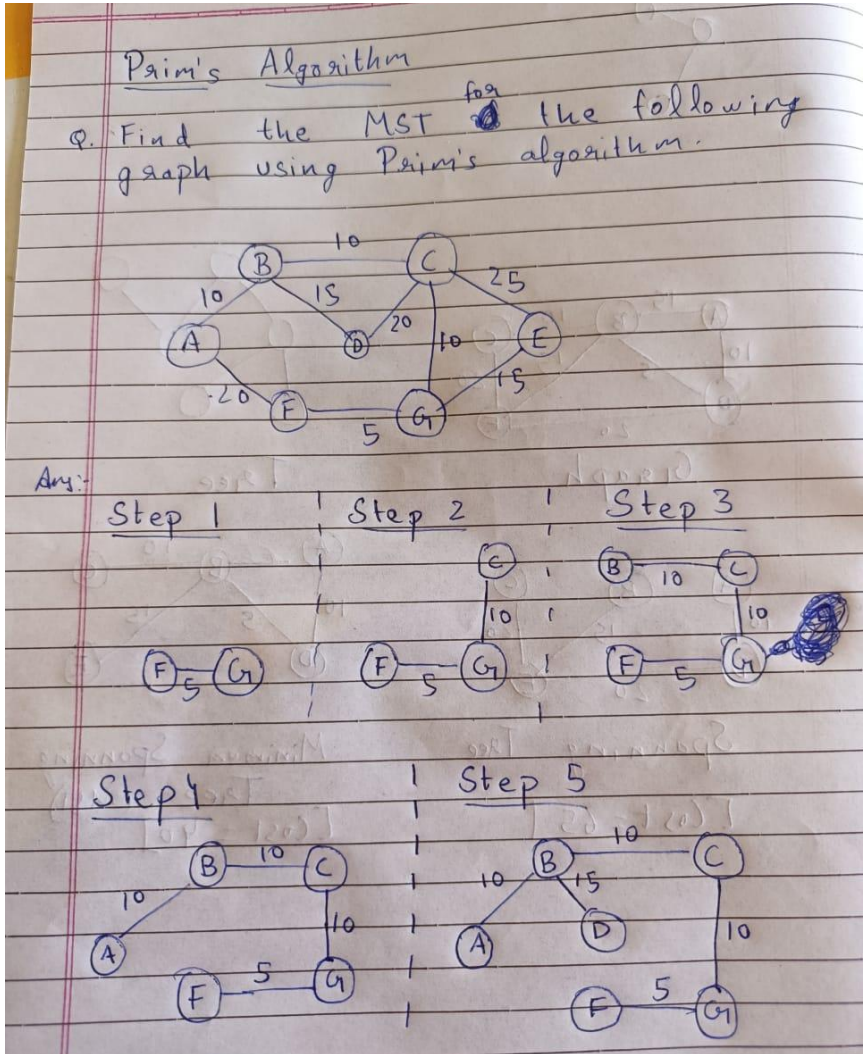
PRIM'S ALGORITHM

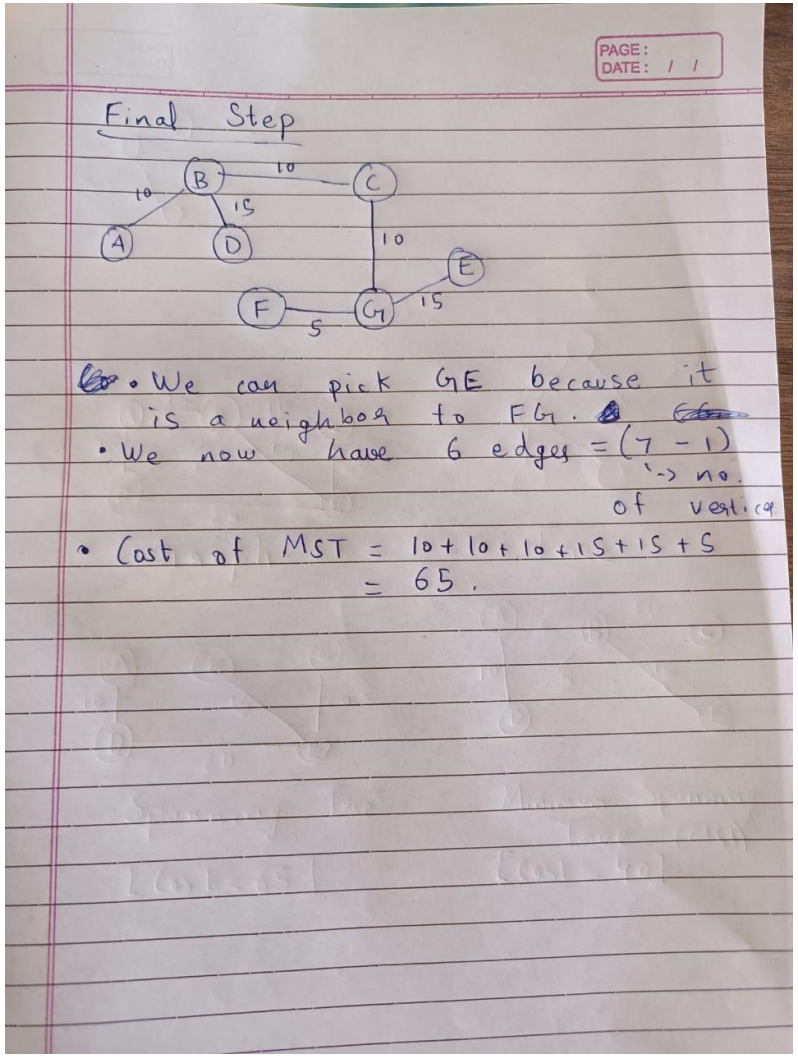
- Steps:
- Create a set *mstSet* that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign the key value as 0 for the first vertex so that it is picked first.
- While mstSet doesn't include all vertices
  - Pick a vertex *u* which is not there in *mstSet* and has a minimum key value.
  - Include *u in the* mstSet.
  - Update the key value of all adjacent vertices of *u*. To update the key values, iterate through all adjacent vertices. For every adjacent vertex *v*, if the weight of edge *u-v* is less than the previous key value of *v*, update the key value as the weight of *u-v*

EXAMPLE

- To solve the algorithm on paper, we can pick the edge with the lowest weight first and start drawing the tree from there.
- Further, we have to look at all the adjacent edges, pick the one with the minimum weight, and draw it.
- Once we have drawn 2 or more edges, we must consider the neighbors of all the 2 or more edges already in the tree, and pick the minimum out of them.
- Repeat the process till we get V-1 edges.

EXAMPLE


Prim's Algorithm
Q. Find the MST for the following graph using Prim's algorithm.
A
B
C
D
E
F
G
10
15
25
20
10
15
20
5
Ans:-
Step 1
Step 2
Step 3
Step 4
Step 5


PAGE:
DATE: / /
Final Step
A
B
C
D
E
F
G
10
15
10
15
5
• We can pick GE because it is a neighbor to FG.
• We now have 6 edges = (7 - 1)
-> no. of vertices
• Cost of MST = 10 + 10 + 10 + 15 + 15 + 5
= 65.

KRUSKAL'S ALGORITHM

- Kruskal's algorithm is also used to find the minimum spanning tree for a graph.
- An MST always has V-1 edges where V is the number of vertices in the graph.
- Steps to follow:

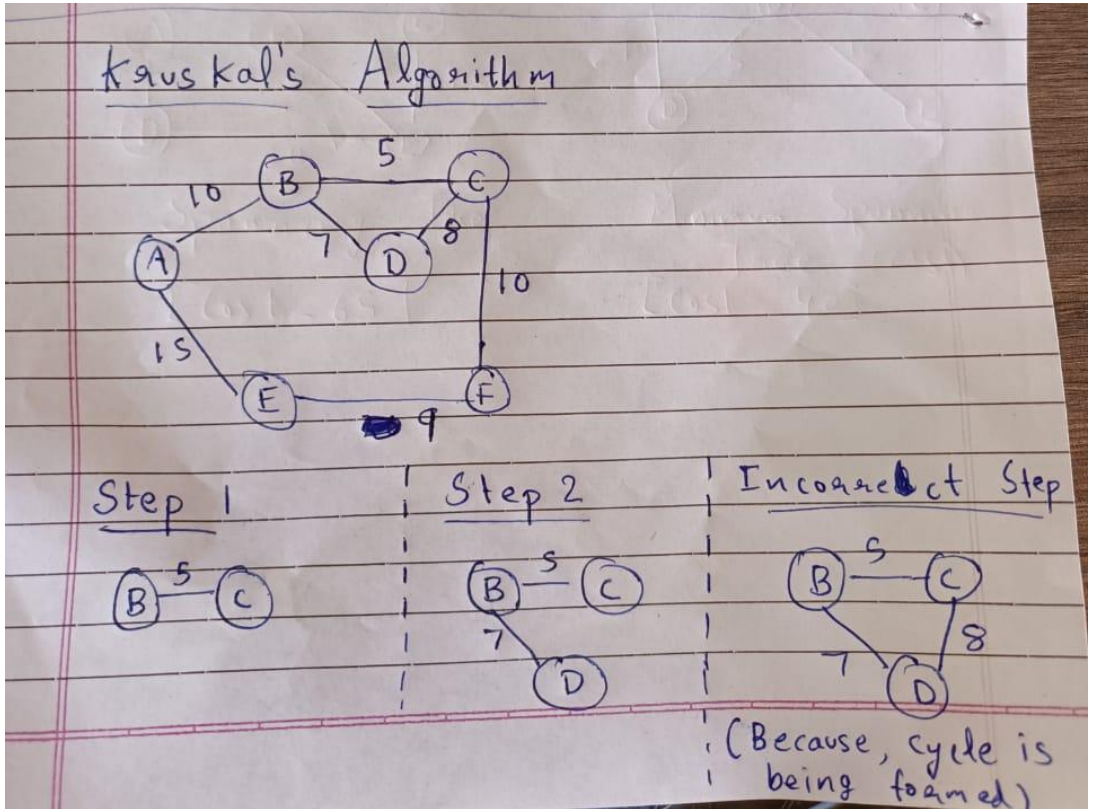
1. Sort all the edges in non-decreasing order of their weight.

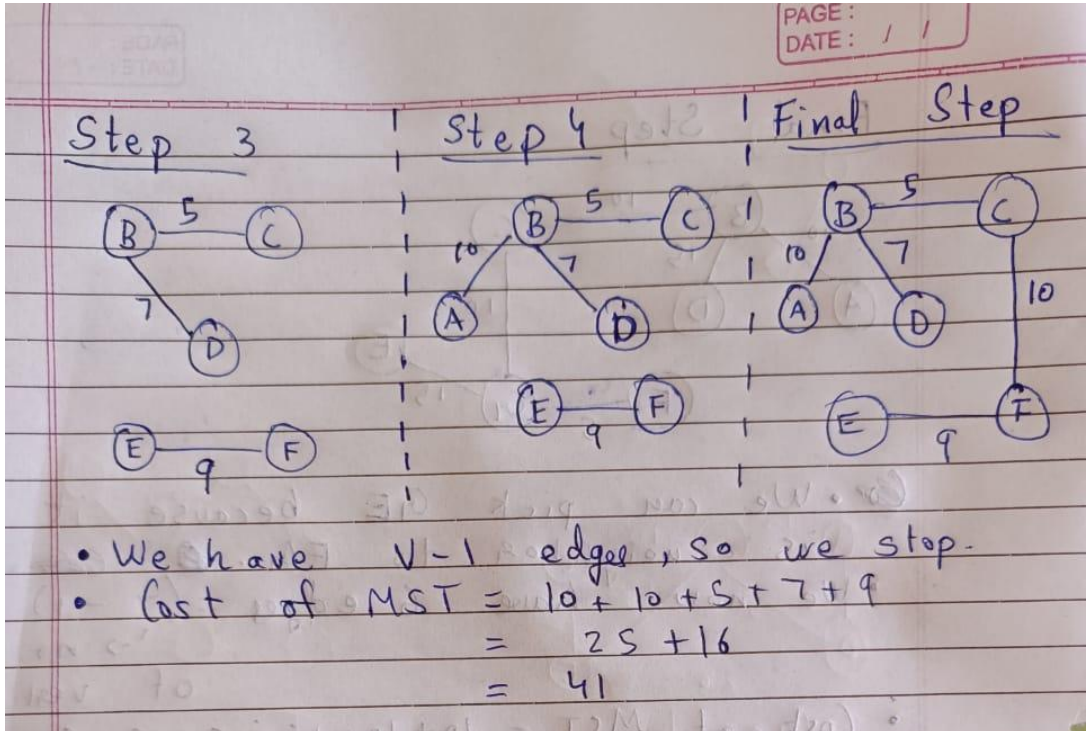
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.

3. Repeat step#2 until there are (V-1) edges in the spanning tree.

- The time complexity of this algorithm is O(Elog(V)) where E is the number of edges in the graph.

EXAMPLE


Kruskal's Algorithm
A
B
C
D
E
F
10
5
7
8
10
15
9
Step 1
Step 2
Incorrect Step
(Because, cycle is being formed)


Step 3
Step 4
Final Step
• We have V-1 edges, so we stop.
• Cost of MST = 10 + 10 + 5 + 7 + 9
= 25 + 16
= 41

DIJKSTRA'S ALGORITHM

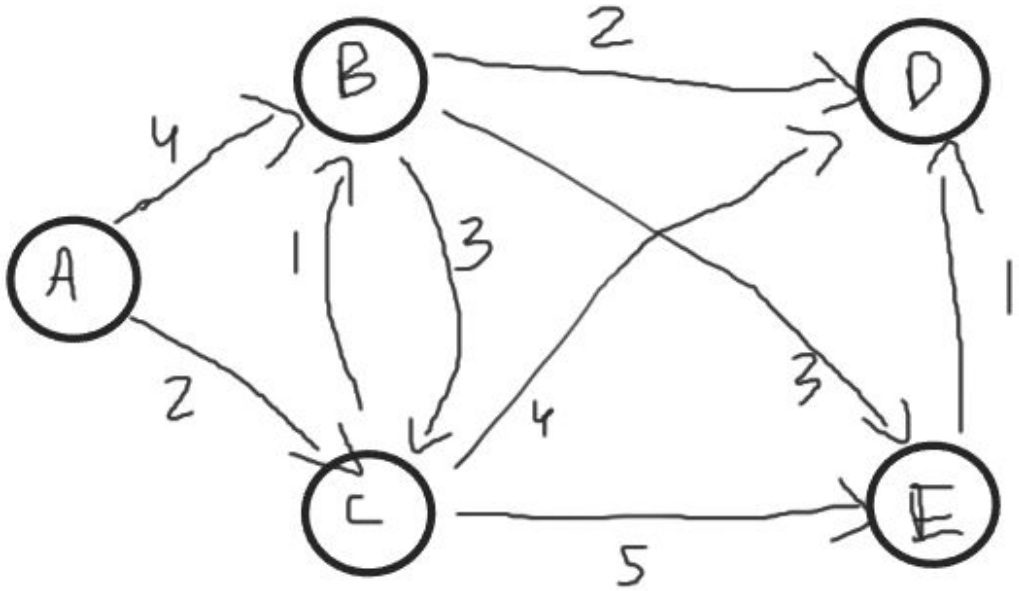
- It is an algorithm to find the shortest path from the source node to all the other nodes in a directed weighted graph.
- It is a greedy algorithm and is therefore fast.
- It is invalid for graphs having negative weights.
- It is invalid for graphs having negative cycles.
- The time complexity of this algorithm is O(Elog(V)) where V is the number of vertices and E is the number of edges in the graph.

STEPS:

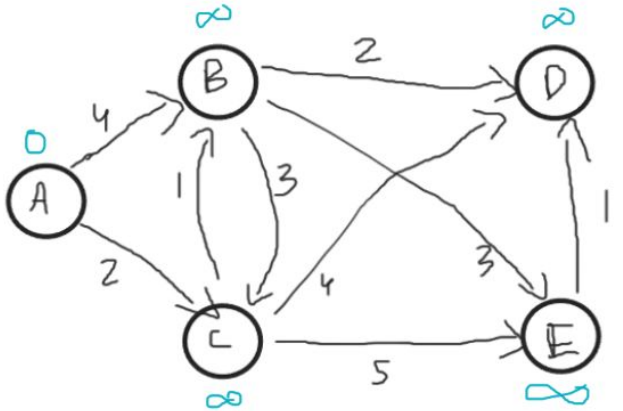
- Create a set **sptSet** (shortest path tree set) that keeps track of vertices included in the shortest-path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.
- Assign a distance value to all vertices in the input graph. Initialize all distance values as **INFINITE**. Assign the distance value as 0 for the source vertex so that it is picked first.
- While **sptSet** doesn't include all vertices
  - Pick a vertex **u** which is not there in **sptSet** and has a minimum distance value.
  - Include u to **sptSet**.
  - Then update distance value of all adjacent vertices of u.
    - To update the distance values, iterate through all adjacent vertices.
    - For every adjacent vertex v, if the sum of the distance value of u (from source) and weight of edge u-v, is less than the distance value of v, then update the distance value of v.

EXAMPLE

Weighted directed graph



Step 1

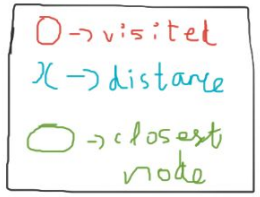


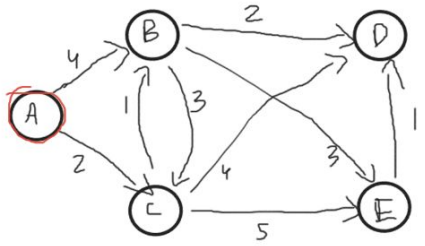
A :
B :
C : ∞
D : ∞
E : ∞

Unvisited Nodes : {A, B, C, D, E}

EXAMPLE

Step 2

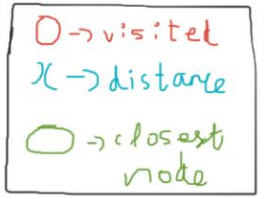


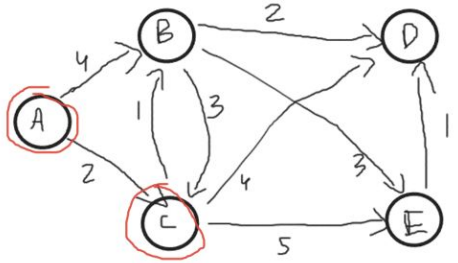


A : 0
B : 4
C : 2
D : ∞
E : ∞

Unvisited Nodes : {~~A~~, B, C, D, E}

Step 3



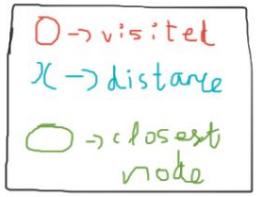


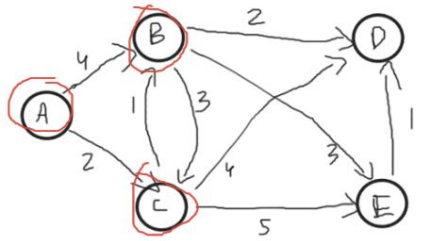
A : 0
B : 2+1 = 3
C : 2
D : 2+4 = 6
E : 2+5 = 7

Unvisited Nodes : {~~A~~, B, ~~C~~, D, E}

EXAMPLE

Step 4

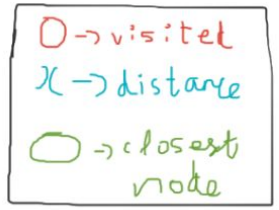


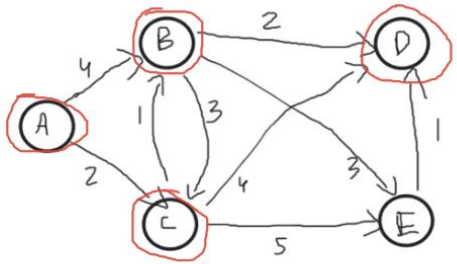


A : 0
B : 3
C : 2
D : 3 + 2 = 5
E : 3 + 3 = 6

Unvisited Nodes : {~~A~~, ~~B~~, ~~C~~, D, E}

Step 5





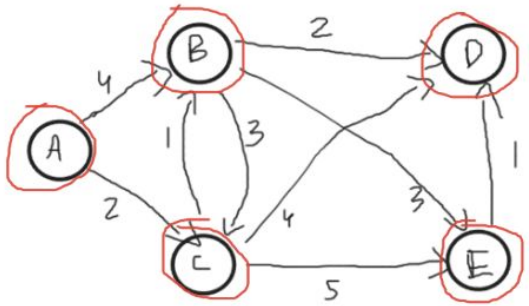
A : 0
B : 3
C : 2
D : 5
E : 6

Unvisited Nodes : {~~A~~, ~~B~~, ~~C~~, ~~D~~, E}

EXAMPLE

Step 6

O -> visited
X -> distance
⬭ -> closest node

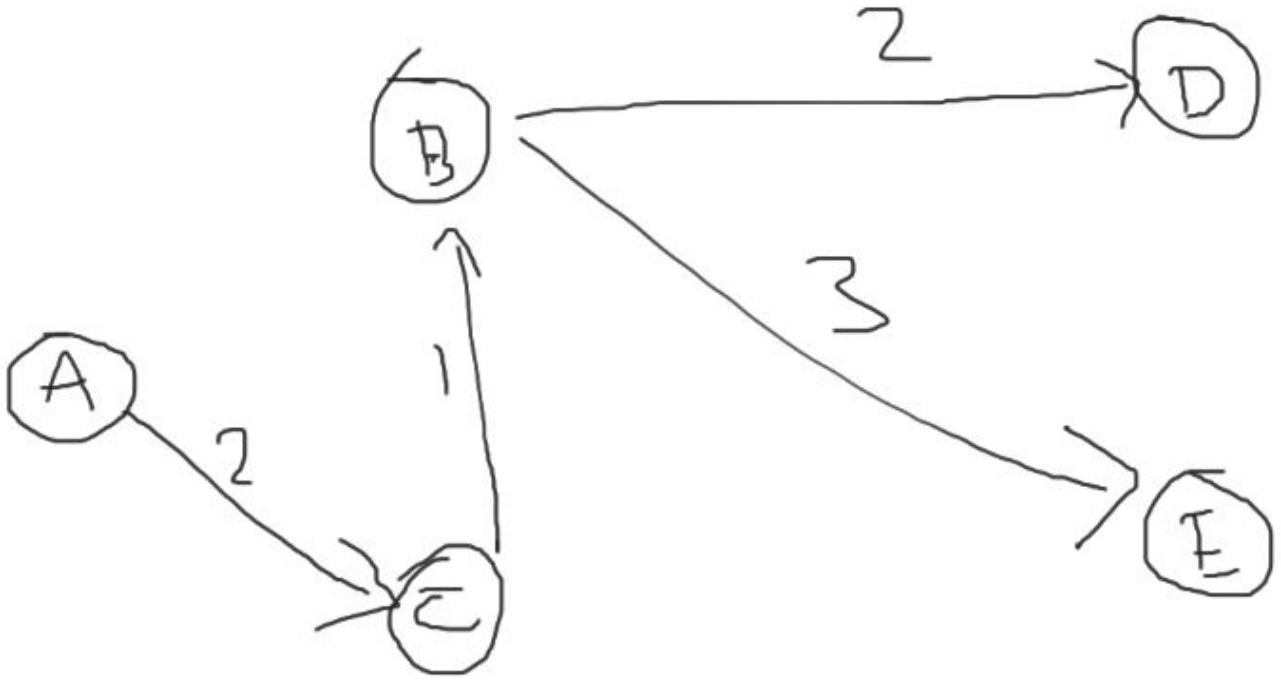


Shortest
A : 0
B : 3
C : 2
D : 5
E : 6

Unvisited Nodes : {~~A~~, ~~B~~, ~~C~~, ~~D~~, ~~E~~}

EXAMPLE

Shortest Path from A to all other nodes



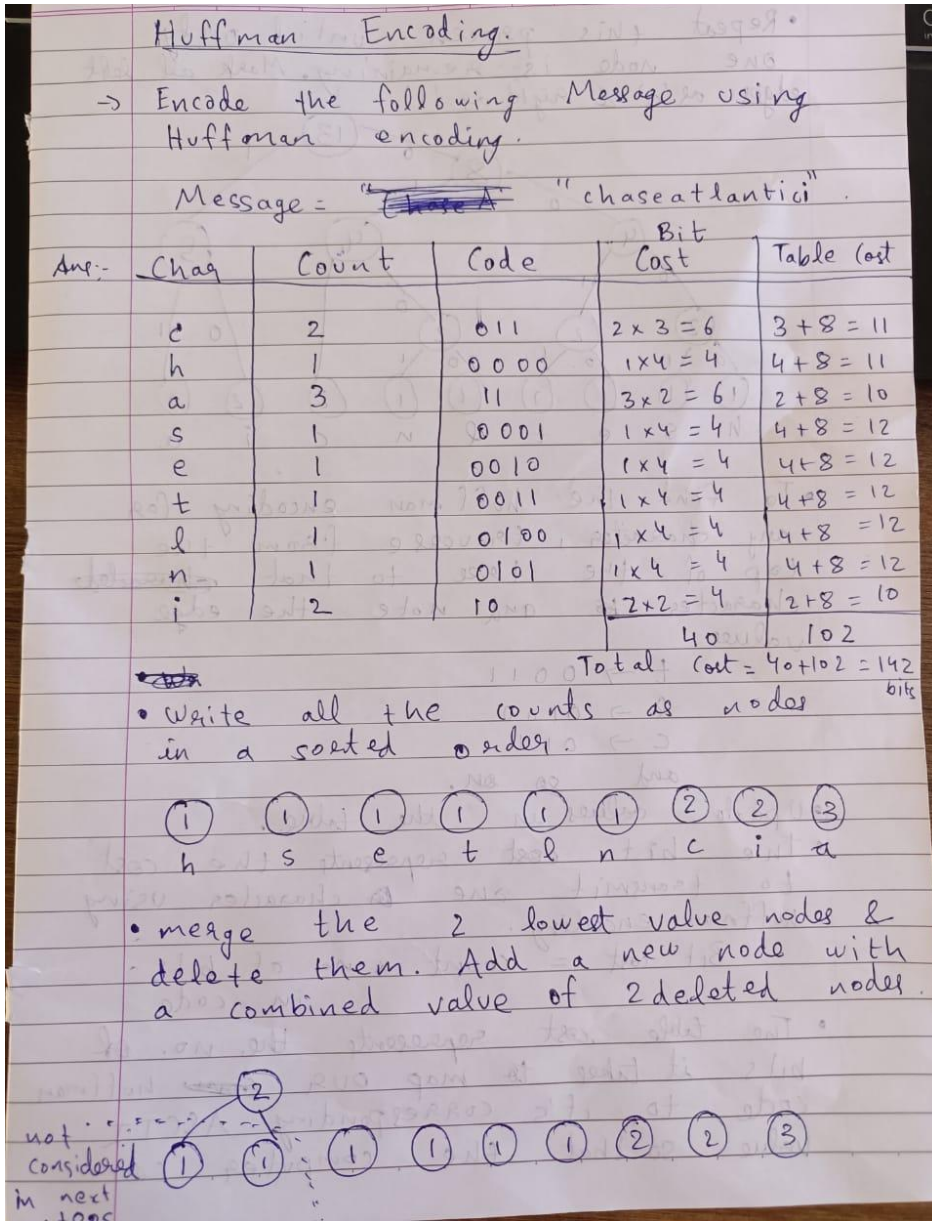
HUFFMAN CODE

- Computer as a machine is incapable of understanding characters and numbers that we use for everyday communication.
- For the computer to be able to process our data, we need to convert it into a binary form.
- If we want to convert a character such as ‘a’ to its binary form, we can do so by using ASCII notation as it is standardized.
- ‘a’ has an ASCII value of 97, which can be written as 01100001 in its binary form.
- Now to convert a message of n characters into its ASCII notation, we have to use 8*n bits as each ASCII code takes up 8 bits.
- This will make our encoded message quite long.

HUFFMAN CODE

- When we are transmitting encoded messages, our goal is to keep the size of the message to a minimum.
- Larger messages take up more space.
- Larger messages take more time to be transmitted hence resulting in latency.
- Larger messages also equate to higher data costs.
- Huffman Encoding is a method by which we can encode our data using **variable-length binary codes** and reduce the size of our encoded message.
- To encode our message using Huffman encoding, we adopt a greedy algorithm with a time complexity of O(nlog(n)).

EXAMPLE



Huffman Encoding.

→ Encode the following Message using Huffman encoding.

Message = "chaseatlantici".

Ans:-

Char	Count	Code	Bit Cost	Table Cost
c	2	011	2×3 = 6	3+8 = 11
h	1	0000	1×4 = 4	4+8 = 11
a	3	11	3×2 = 6	2+8 = 10
s	1	0001	1×4 = 4	4+8 = 12
e	1	0010	1×4 = 4	4+8 = 12
t	1	0011	1×4 = 4	4+8 = 12
l	1	0100	1×4 = 4	4+8 = 12
n	1	0101	1×4 = 4	4+8 = 12
i	2	10	2×2 = 4	2+8 = 10
			40	102

Total Cost = 40+102 = 142 bits

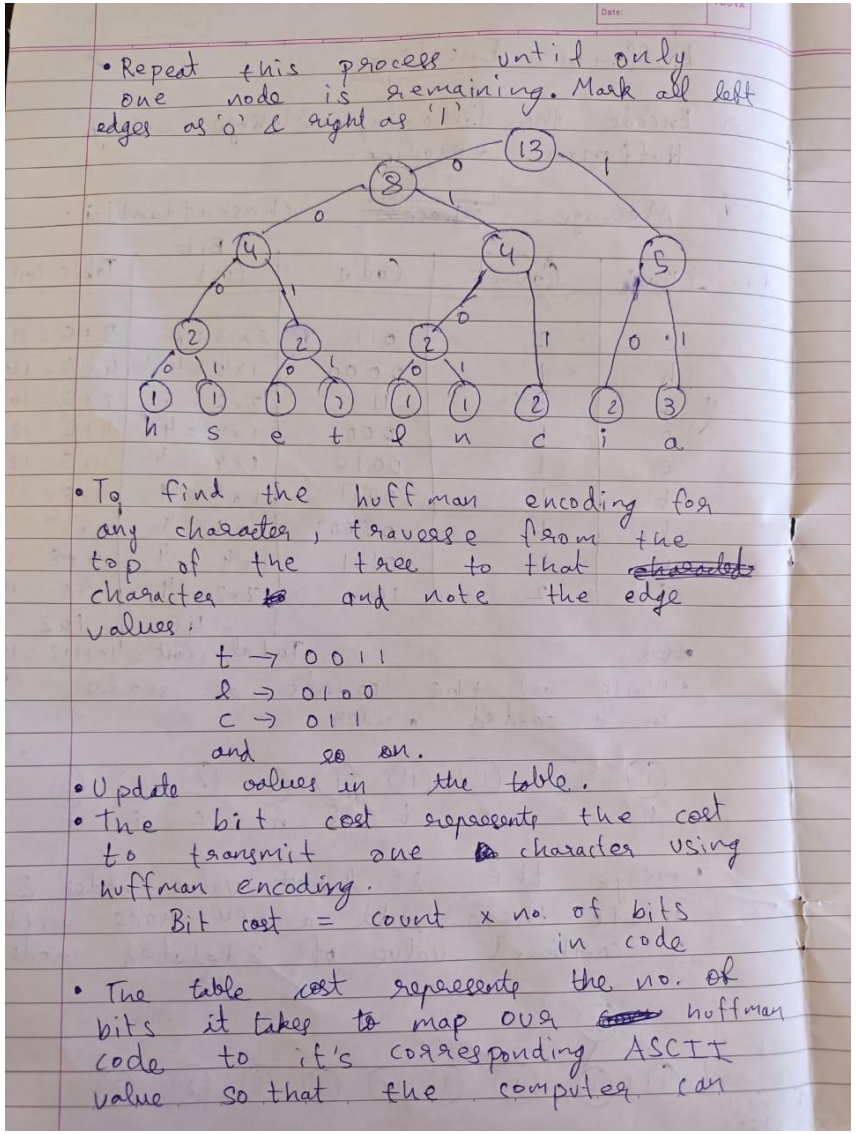
- Write all the counts as nodes in a sorted order.

(1) (1) (1) (1) (1) (1) (2) (2) (3)
h s e t l n c i a

- merge the 2 lowest value nodes & delete them. Add a new node with a combined value of 2 deleted nodes.

(2)
not considered in next
(1) (1) (1) (1) (1) (1) (2) (2) (3)

EXAMPLE



- Repeat this process until only one node is remaining. Mark all left edges as '0' & right as '1'.

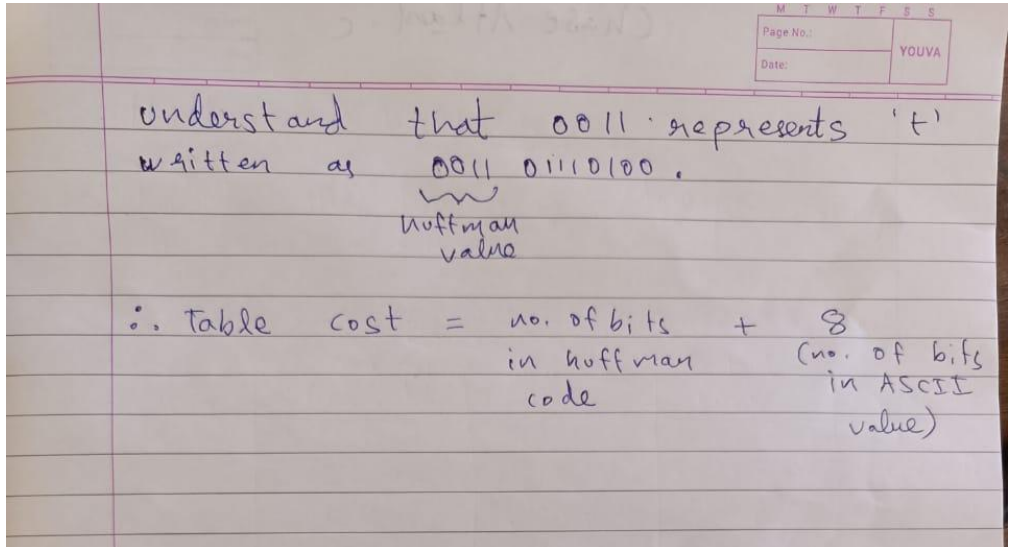
- To find the huffman encoding for any character, traverse from the top of the tree to that character and note the edge values:

  t → 0011
  l → 0100
  c → 011
  and so on.

- Update values in the table.
- The bit cost represents the cost to transmit one character using huffman encoding.

  Bit cost = count × no. of bits in code

- The table cost represents the no. of bits it takes to map our huffman code to it's corresponding ASCII value so that the computer can



understand that 0011 represents 't' written as 0011 01110100.
(0011: huffman value)

∴ Table cost = no. of bits in huffman code + 8 (no. of bits in ASCII value)

ACTIVITY SELECTION PROBLEM

Problem statement:

- You are given n activities along with the starting and finishing times of those activities. You have to complete the maximum number of activities with the following conditions in mind:

  - You can only do one activity at a time.

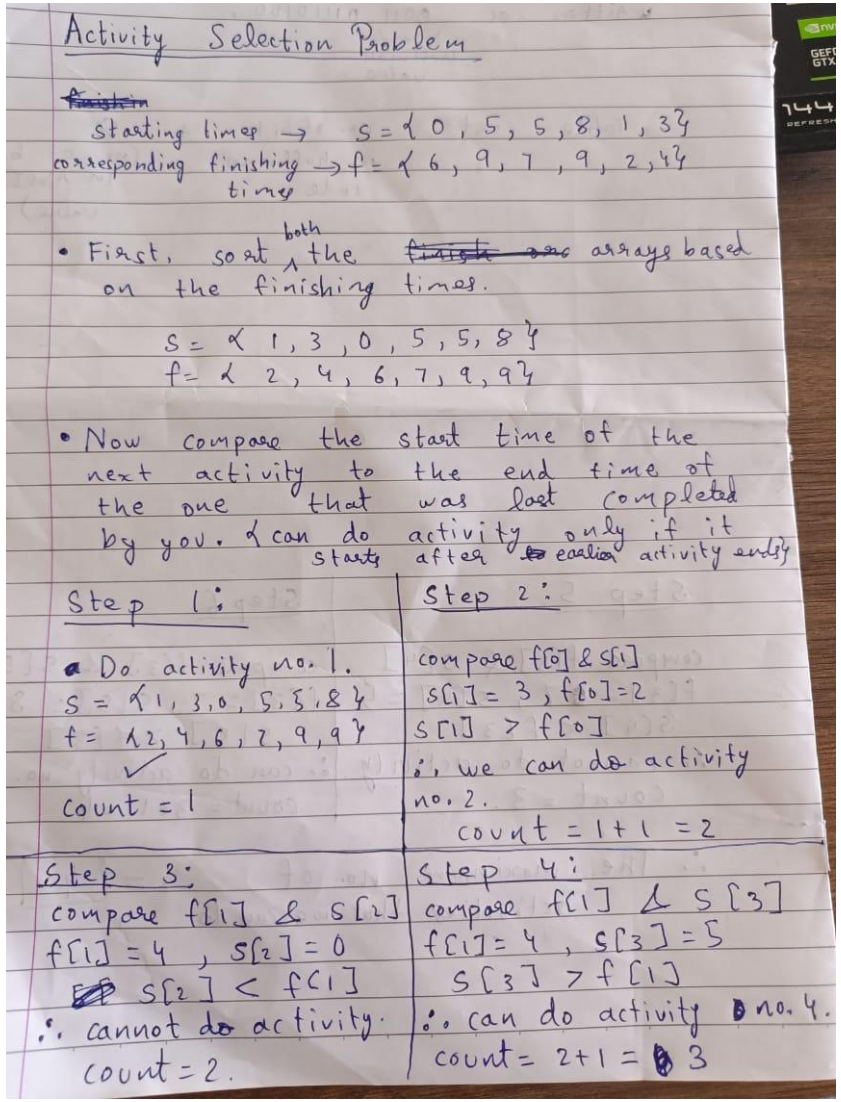
  - Once you start with an activity, you have to finish it completely before moving to the next activity.

- You have to pick an order that allows you to do the most number of activities.

ACTIVITY SELECTION PROBLEM

- At first glance, one might be tempted to pick the activity with the minimum starting time and finish that first.
- The starting time, however, gives us no idea about how long the activity is going to last.
- On the other hand, the ending time of an activity tells us how soon the activity can be finished.
- The sooner we finish one activity, the sooner we can move on to the next activity. This will allow us to do the most number of activities in the given time period.
- Therefore, the strategy to solve this problem is to sort the activities according to their finishing time and then do the activities with the least finishing times.
- Time complexity if the activities are already sorted according to finish time: O(n)
- Time complexity if the activities are not sorted according to finish time: O(nlog(n))

EXAMPLE



Activity Selection Problem

starting times → $s = \{0, 5, 5, 8, 1, 3\}$
corresponding finishing times → $f = \{6, 9, 7, 9, 2, 4\}$

- First, sort both the arrays based on the finishing times.

$s = \{1, 3, 0, 5, 5, 8\}$
$f = \{2, 4, 6, 7, 9, 9\}$

- Now compare the start time of the next activity to the end time of the one that was last completed by you. I can do activity only if it starts after earlier activity ends

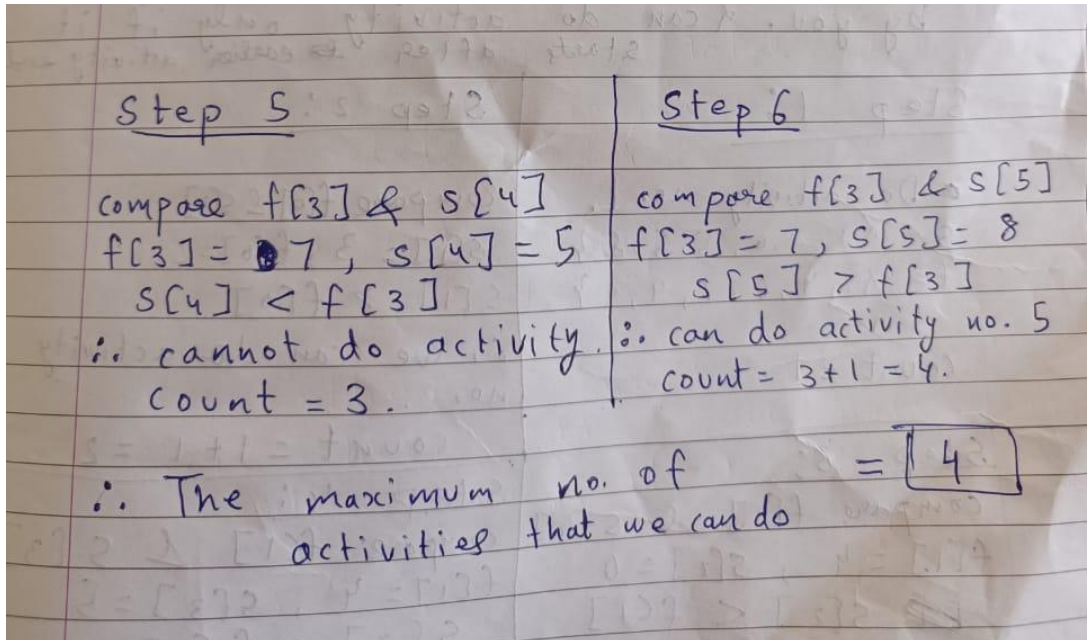
Step 1:
- Do activity no. 1.

$s = \{1, 3, 0, 5, 5, 8\}$
$f = \{2, 4, 6, 7, 9, 9\}$ ✓
count = 1

Step 2:
compare f[0] & s[1]
s[1] = 3, f[0] = 2
s[1] > f[0]
∴ we can do activity no. 2.
count = 1 + 1 = 2

Step 3:
compare f[1] & s[2]
f[1] = 4, s[2] = 0
s[2] < f[1]
∴ cannot do activity.
count = 2.

Step 4:
compare f[1] & s[3]
f[1] = 4, s[3] = 5
s[3] > f[1]
∴ can do activity no. 4.
count = 2 + 1 = 3



Step 5
compare f[3] & s[4]
f[3] = 7, s[4] = 5
s[4] < f[3]
∴ cannot do activity.
count = 3.

Step 6
compare f[3] & s[5]
f[3] = 7, s[5] = 8
s[5] > f[3]
∴ can do activity no. 5
count = 3 + 1 = 4.

∴ The maximum no. of activities that we can do = 4